

Projeto e Análise de Algoritmos

Notação assintótica

Thiago Pinheiro de Araújo

EMAp/FGV

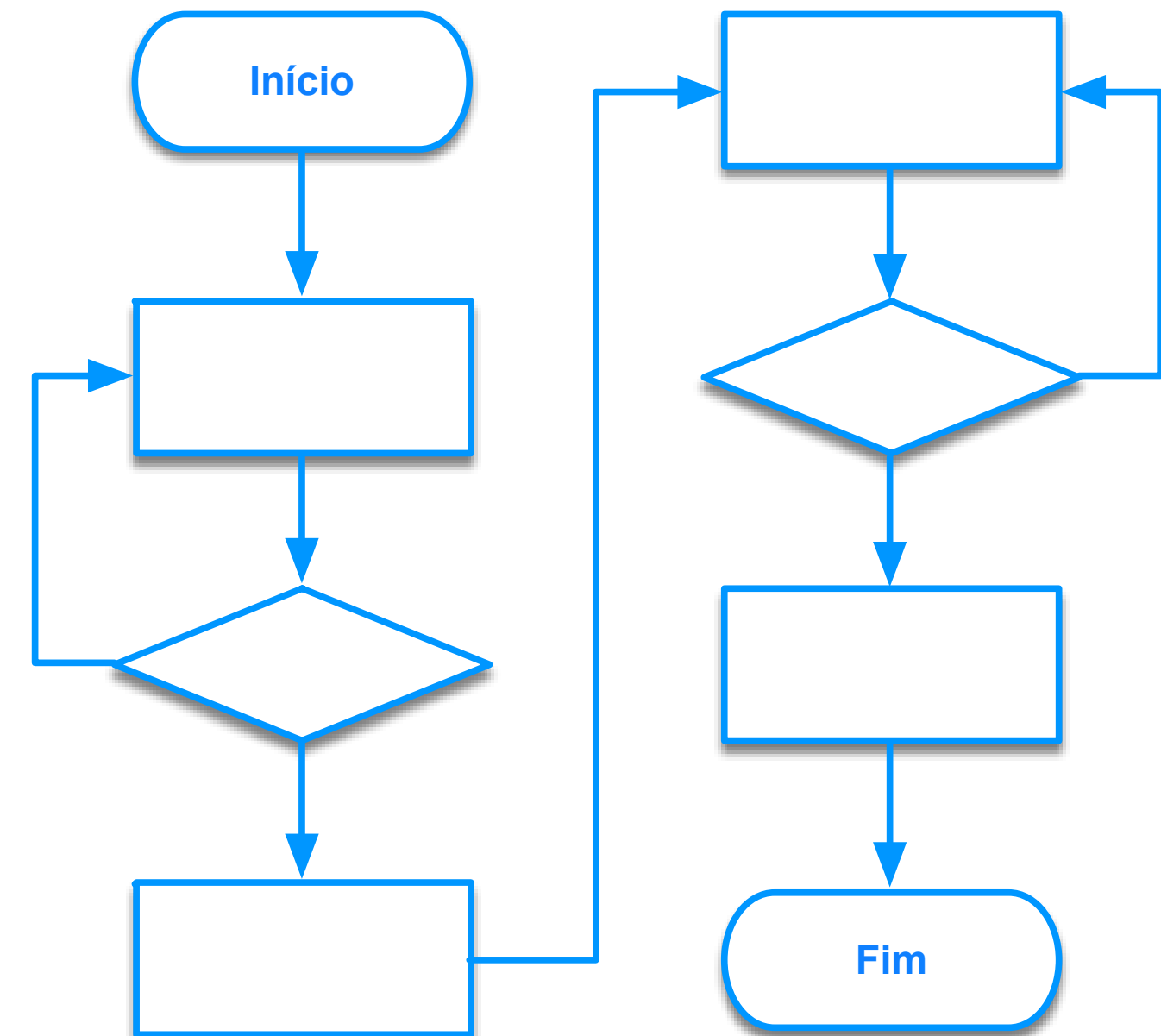
2025.2

Discutir como avaliar a complexidade e
apresentar a **notação assintótica** como
solução para analisar e comparar algoritmos

Porque estudar esse assunto?

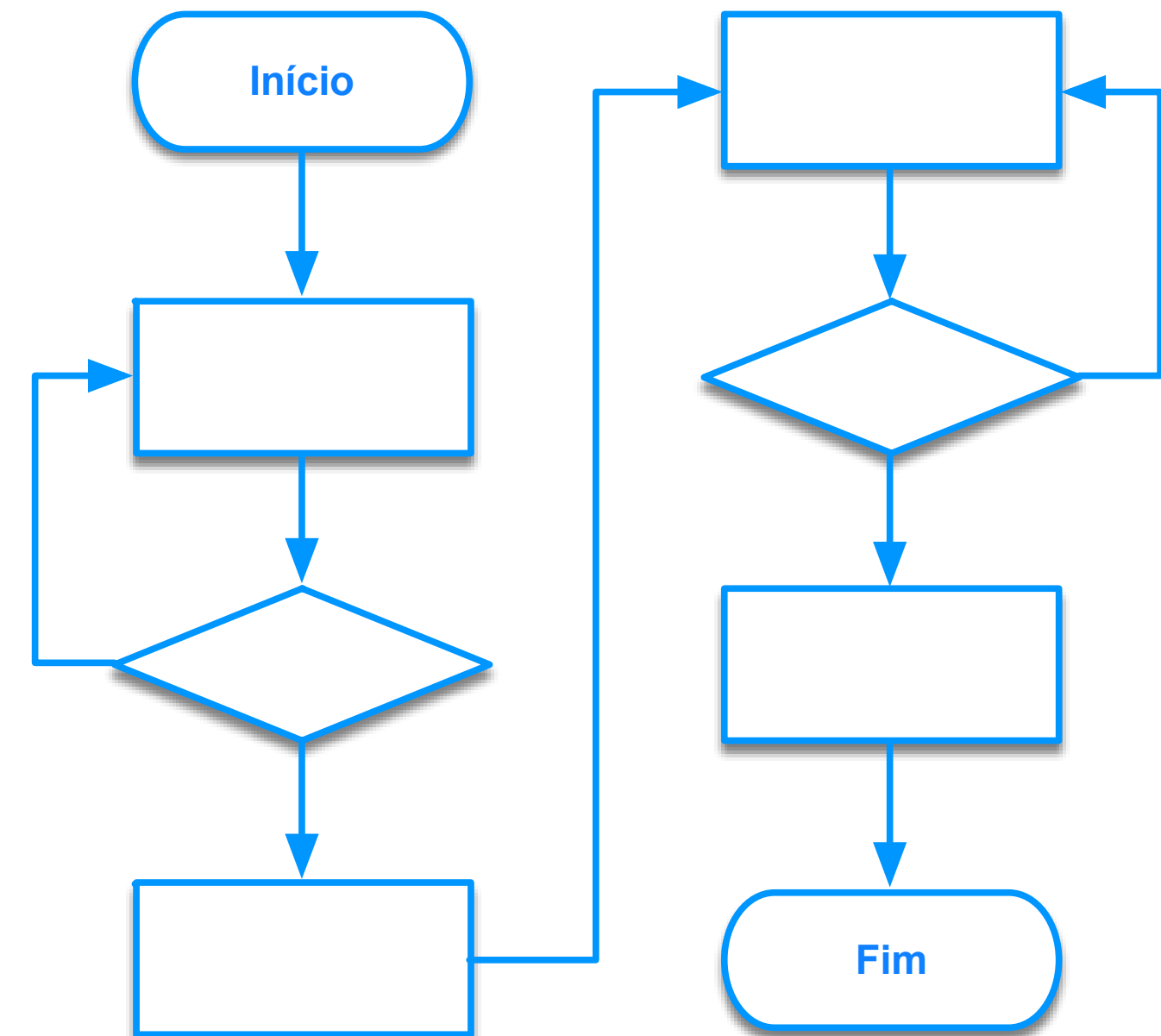
- Estimar como um algoritmo irá se comportar com volumes grandes de dados.
- Comparar a eficiência de algoritmos diferentes.
- Escolher algoritmos adequados para solucionar problemas.
- Entender as limitações dos algoritmos escolhidos.

- O que é um **algoritmo**?
 - É uma sequência de passos que podem ser computados com o objetivo de solucionar um problema.
- Usualmente empregamos o termo para problemas não-triviais.

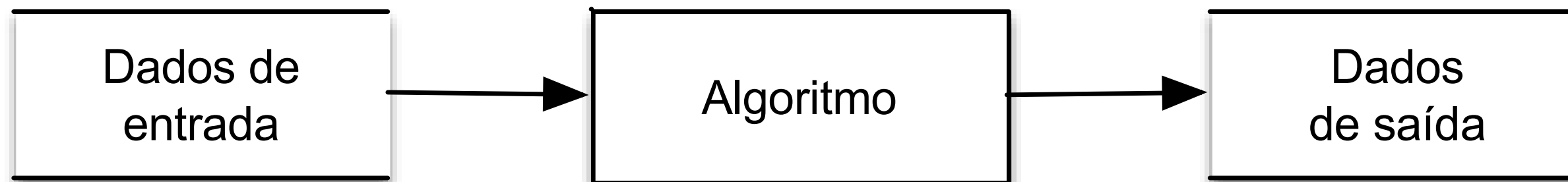


Definições

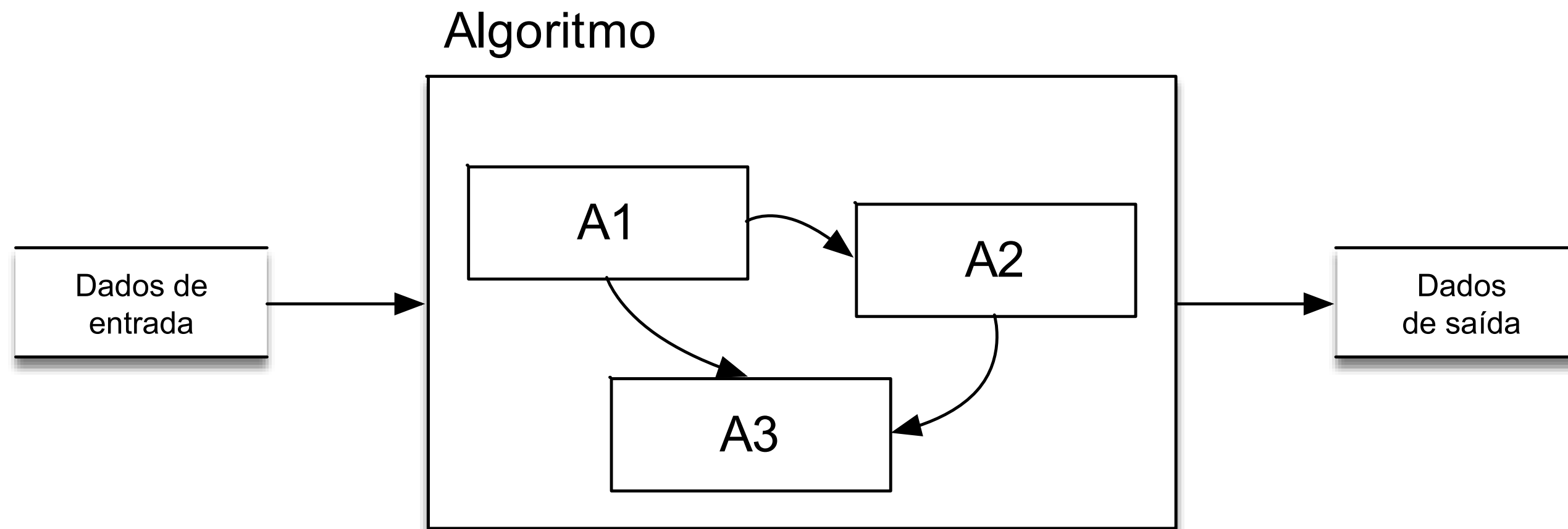
- Um problema pode apresentar mais de uma solução
 - Ou seja, diferentes algoritmos podem ser projetados para resolver o mesmo problema.
- Qual seria o propósito?
 - Estratégias de consumo de recursos.
 - Precisão.
 - Compreensibilidade.
 - Facilidade de implementação.
 - Etc.



- A **entrada de um algoritmo** é o conjunto de dados submetido à uma execução do mesmo
 - O tamanho da entrada consiste no número de elementos contido no conjunto.
- A **saída de um algoritmo** é o conjunto de dados resultante de uma execução, relacionado diretamente com a sua entrada.



- A solução computacional de um problema (um algoritmo) pode ser composta pela combinação de outros algoritmos.



Análise da complexidade

Cenário para contextualização

1. Suponha que você acabou de desenvolver um algoritmo.
2. Realizou a implementação e implantou no ambiente de produção.
3. No entanto, identificou um desempenho muito baixo para entradas grandes.
4. Você decide criar um novo algoritmo com maior desempenho.

Definição de desempenho

- **Primeira pergunta:** O que define o desempenho?
- **Resposta (errada):** Podemos calcular o tempo que o algoritmo leva para computar essa entrada grande
 - Ok, mas e se o comportamento for diferente para diferentes entradas?
- **Resposta:** Podemos encontrar uma função $T(n)$ que defina o tempo de execução com base no tamanho da entrada
 - Corresponde ao tempo que o algoritmo leva para processar uma entrada de tamanho n .

Análise da complexidade

Definição de tempo

- Considere o seguinte algoritmo:
- Quanto tempo levaria para computar uma entrada com 1 bilhão de números?
 - Computador de alto desempenho: 485 segundos
 - Computador de propósito geral: 662 segundos
 - Computador embarcado: 2239 segundos
- O tempo varia de acordo com:
 - A arquitetura do computador.
 - A linguagem de programação.
 - O compilador.

```
def select_even(numbers):  
    result = []  
    for number in numbers:  
        if number % 2 == 0:  
            result.append(number)  
    return result
```

Conclusão: a unidade temporal em segundos não é adequada para avaliar o desempenho de algoritmos.

Definição de tempo

- Poderíamos contar o número de linhas executadas?
 - Também não, pois linhas podem agrupar múltiplas instruções.
 - Exemplo: qual seria a diferença de tempo entre os trechos de código a seguir?

```
def select_even1(numbers):  
    result = []  
    for number in numbers:  
        mod = number % 2  
        is_even = mod == 0  
        if is_even:  
            result.append(number)  
    return result
```

7 linhas

```
def select_even2(numbers):  
    return [number for number in  
            numbers if number % 2 == 0]
```

1 linha

- Solução: definir a complexidade baseado no número de instruções executadas.
- Considere o seguinte algoritmo:

```
def sum(numbers):  
    result = 0  
    for number in numbers:  
        result += number  
    return result
```

- Quantas instruções são executadas para a entrada [29, 7, 12]?
 - Resposta: $1 + 3 * 2 = 7$

- **Problema:** o número de instruções executadas pode variar dependendo dos dados de entrada.
 - Considere novamente o algoritmo ao lado.
- Quantas instruções são executadas para a entrada [2, 4, 8]?
 - Resposta: $1 + 3 * 3 = 10$
- E para a entrada [1, 4, 7]?
 - Resposta: $1 + 3 * 2 + 1 = 8$

```
def select_even(numbers):  
    result = []  
    for number in numbers:  
        if number % 2 == 0:  
            result.append(number)  
    return result
```

Conclusão: contagem diferente, para o mesmo tamanho de entrada!

Contagem de instruções

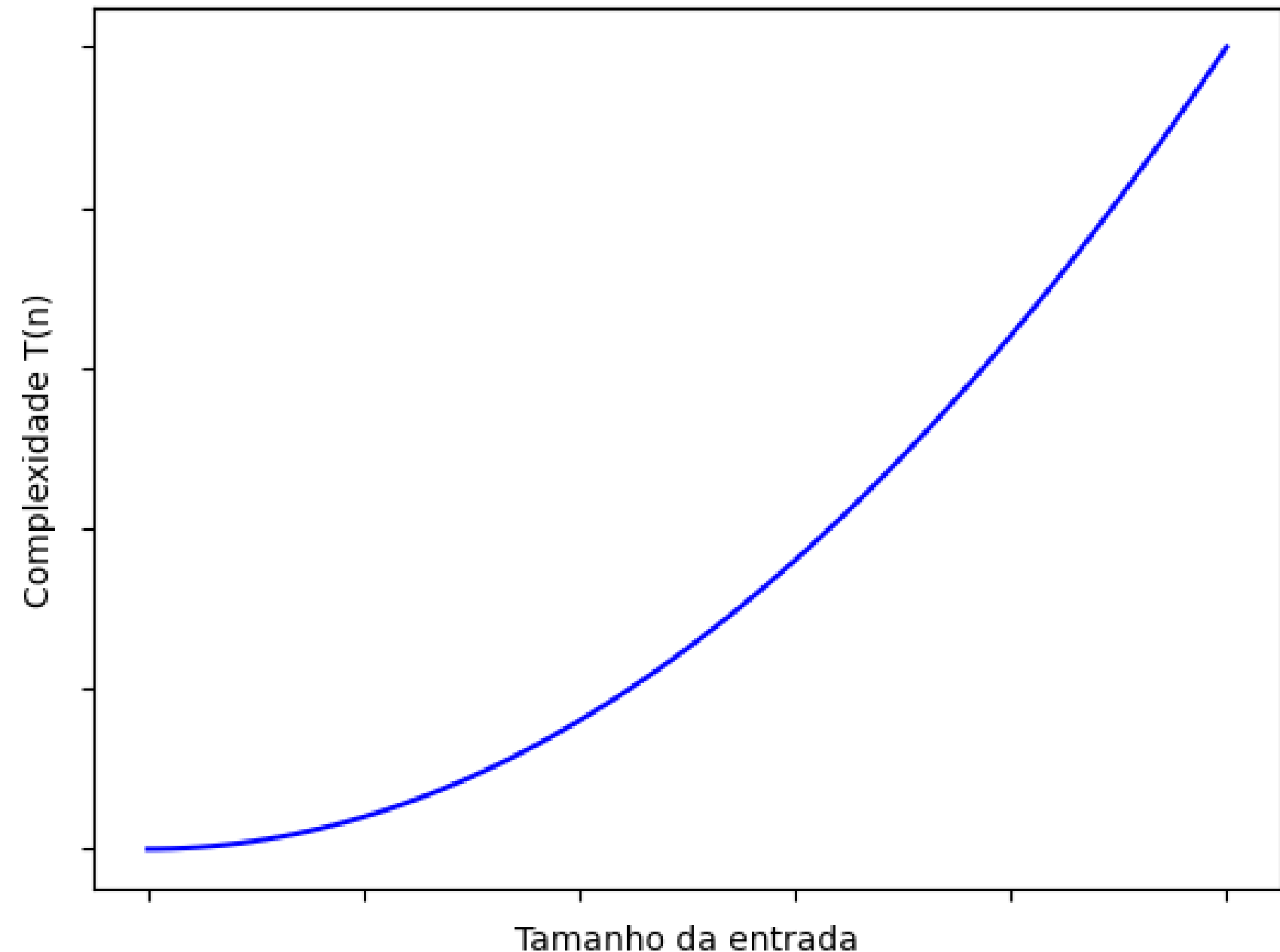
- **Questão:** considerando todas as entradas possíveis de tamanho n para um algoritmo, qual escolher para avaliar a complexidade?
- Resposta: a instância que implicar na execução do maior número de instruções possível.
- É uma visão pessimista, porém avalia a eficiência no pior caso
 - Falaremos mais sobre isso...

Análise da complexidade

Definição da função de complexidade

- Portanto, podemos definir a complexidade de um determinado algoritmo através de uma função como:

$T(n)$ = **número de instruções**
executadas considerando
a instância que exercita o
pior caso.



Definição da função de complexidade

- **Exemplo 1:** qual função define a complexidade do algoritmo abaixo?

```
def sum(numbers):  
    result = 0  
    for number in numbers:  
        result += number  
    return result
```

- Resposta: $T(n) \approx 2n + 1$
 - Inicialização da variável *result* uma única vez (+1).
 - Soma e atribuição (+2) de cada elemento da lista (n vezes).
 - Desconsideramos:
 - A chamada do método.
 - Instruções internas do loop.
 - Retorno do resultado.

Definição da função de complexidade

■ Exemplo 2: qual função define a complexidade do algoritmo abaixo?

```
def do_something(numbers):  
    intermediate = []  
    for number in numbers:  
        intermediate.append(number * number)  
  
    result = []  
    for number_1 in intermediate:  
        parcial_result = []  
        for number_2 in intermediate:  
            parcial_result.append(number_1 * number_2)  
  
        result.append(parcial_result)  
  
    return result
```

** considere a invocação da função append como uma única instrução.*

■ Resposta: $T(n) \approx 2n^2 + 4n + 2$

Definição da função de complexidade

- **Exemplo 3:** qual função define a complexidade do algoritmo abaixo?

```
def compute(votes):  
    sum = 0  
    for i, vote in enumerate(votes):  
        sum += vote  
  
        if i % 10 == 0:  
            print(f'Parcial result: {sum}')  
    return sum
```

* considere a invocação da função print como uma única instrução.

** desconsidere a invocação da função enumerate.

- Resposta: $T(n) \approx \frac{41n}{10} + 1$

Definição da função de complexidade

- **Exemplo 4:** qual função define a complexidade do algoritmo abaixo?

```
def sum_values(numbers):  
    sum_total = 0  
    sum_ignore_first_100 = 0  
    for i, number in enumerate(numbers):  
        sum_total += number  
        if i > 100:  
            sum_ignore_first_100 += number  
  
    return sum_total
```

** desconsidere a invocação da função enumerate.*

- Resposta: $T(n) \approx 5n - 198$

Notação assintótica

Motivação

- Determinar com precisão a função que descreve a complexidade de um algoritmo pode ser uma tarefa difícil e muito trabalhosa
 - Considere que em um projeto real os algoritmos serão muito maiores e mais complexos que os exemplos dos slides anteriores.
- Tais algoritmos podem requerer um tempo considerável para determinar a função de complexidade
 - A precisão obtida pode se tornar irrelevante ou mesmo induzir à conclusões erradas.

Motivação

- Por exemplo, considere um algoritmo cuja função de complexidade é definida por:
 - $T(n) = 2n^2 - 50n$
- Você conseguiria garantir que ele seria mais rápido que um algoritmo cuja complexidade é definida pela função a seguir?
 - $T(n) = 3n^2 + 100n$

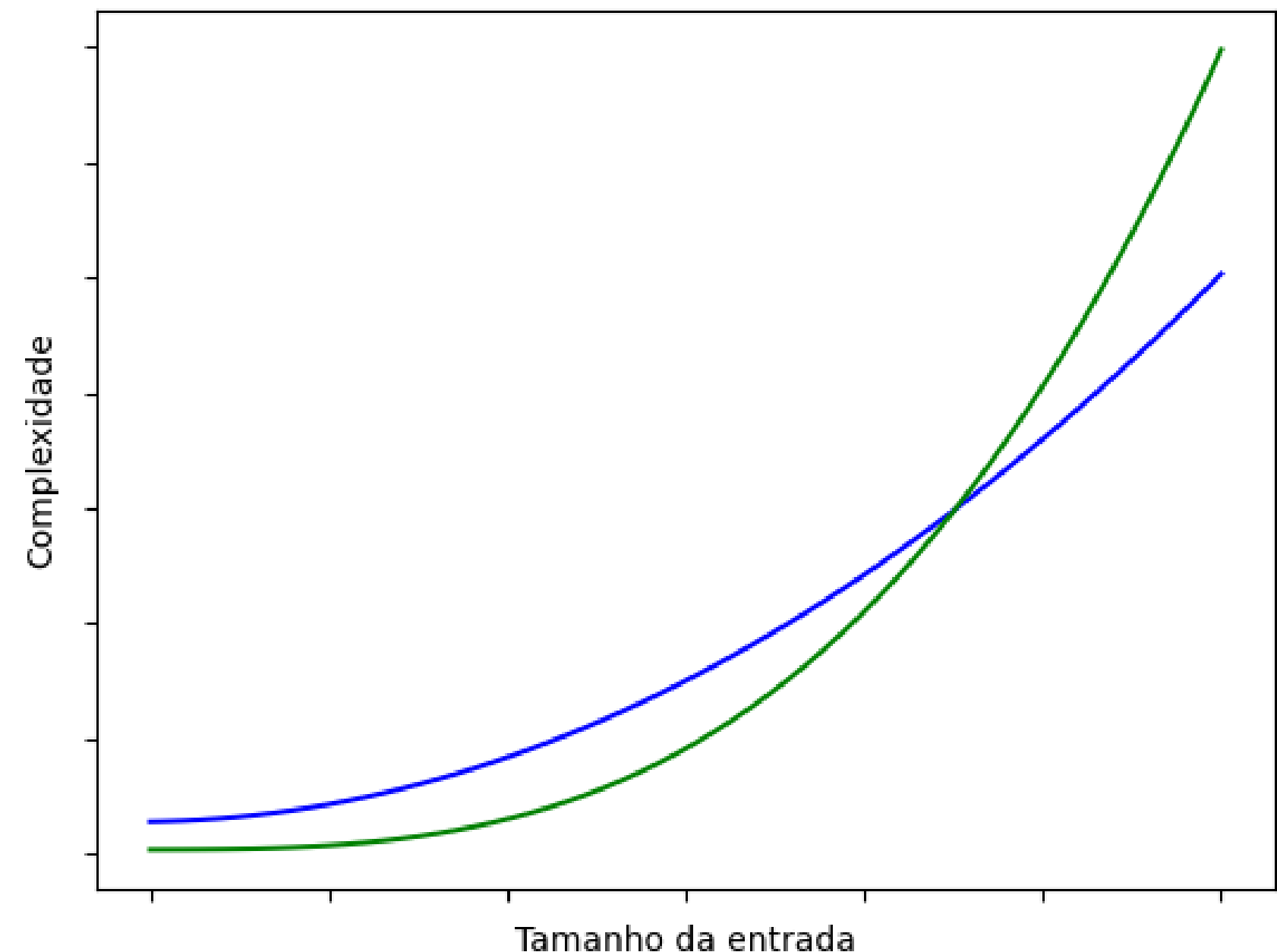
A resposta é não!

Considere que as instruções do primeiro algoritmo podem apresentar maior esforço computacional (individualmente).

Notação assintótica

Motivação

- No entanto, podemos observar que a diferença entre o grau das funções impacta diretamente na taxa de crescimento das funções
- Ex: Um algoritmo com complexidade n^3 cresce mais rápido que um algoritmo n^2
 - $T_1(n) = 8n^2 + 3n + 70$
 - $T_2(n) = n^3 + 0.5n + 10$
- Na prática, queremos apenas poder **comparar** a ordem assintótica de crescimento.

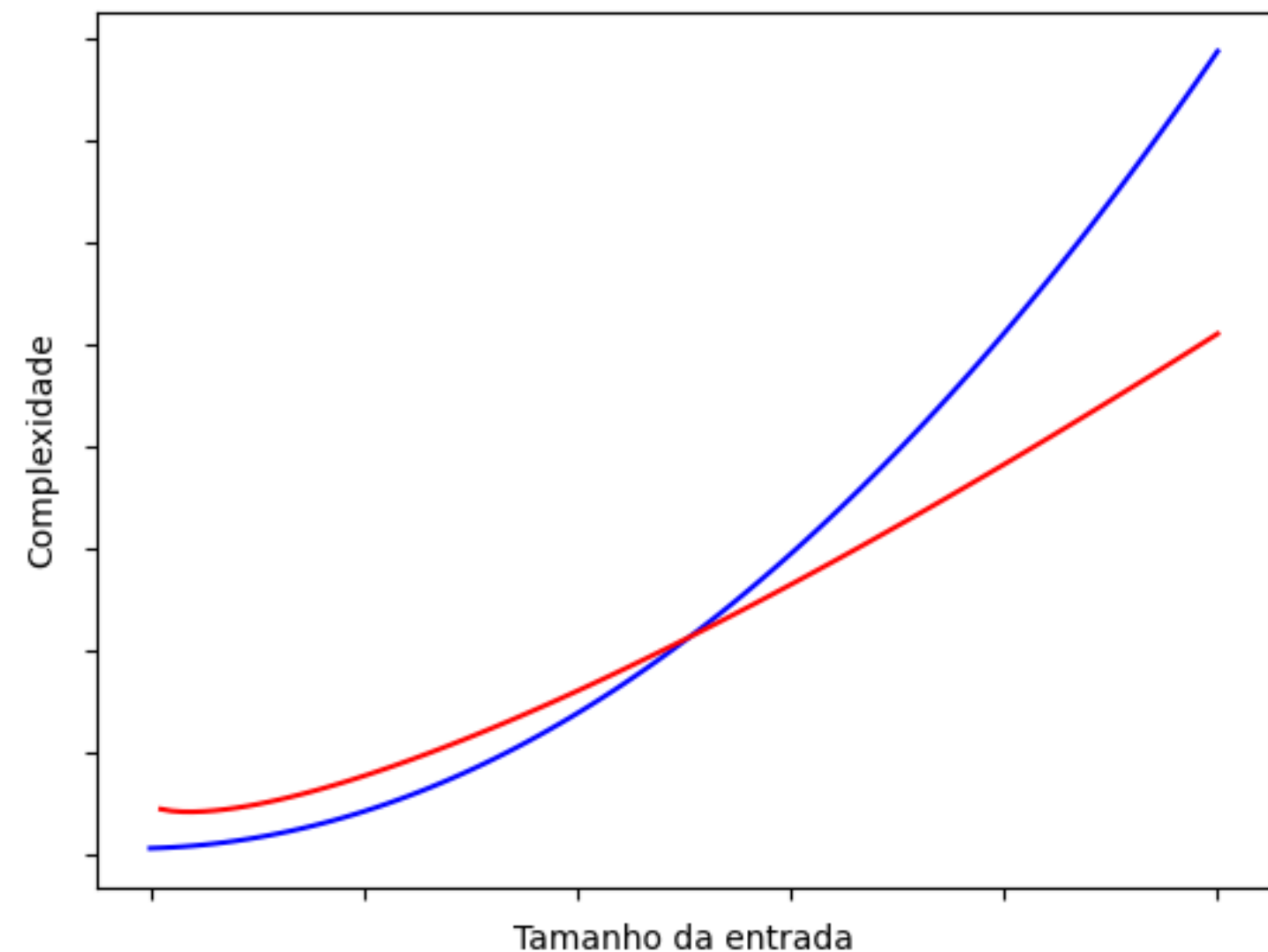


- Portanto, o nosso objetivo durante uma análise será encontrar a **ordem assintótica de crescimento** da função $T(n)$
 - Ao invés de buscar a função precisa que descreve a complexidade do algoritmo.
- Por exemplo, um algoritmo cuja complexidade seja definida pela função:
 - $T(n) = 10n^3 + 7n^2 + 1200$
- Será avaliado como:
 - $T(n) = \Theta(n^3)$

Notação assintótica

Ordem de crescimento

- A **ordem assintótica de crescimento** de $T(n)$ é definida encontrando as funções que descrevem os limites superior e inferior de $T(n)$.
- **Notação O:**
 - Informal: $T(n)$ pertence ao conjunto $O(f(n))$ se $T(n)$ crescer no máximo na mesma ordem de grandeza de $f(n)$.



Notação assintótica

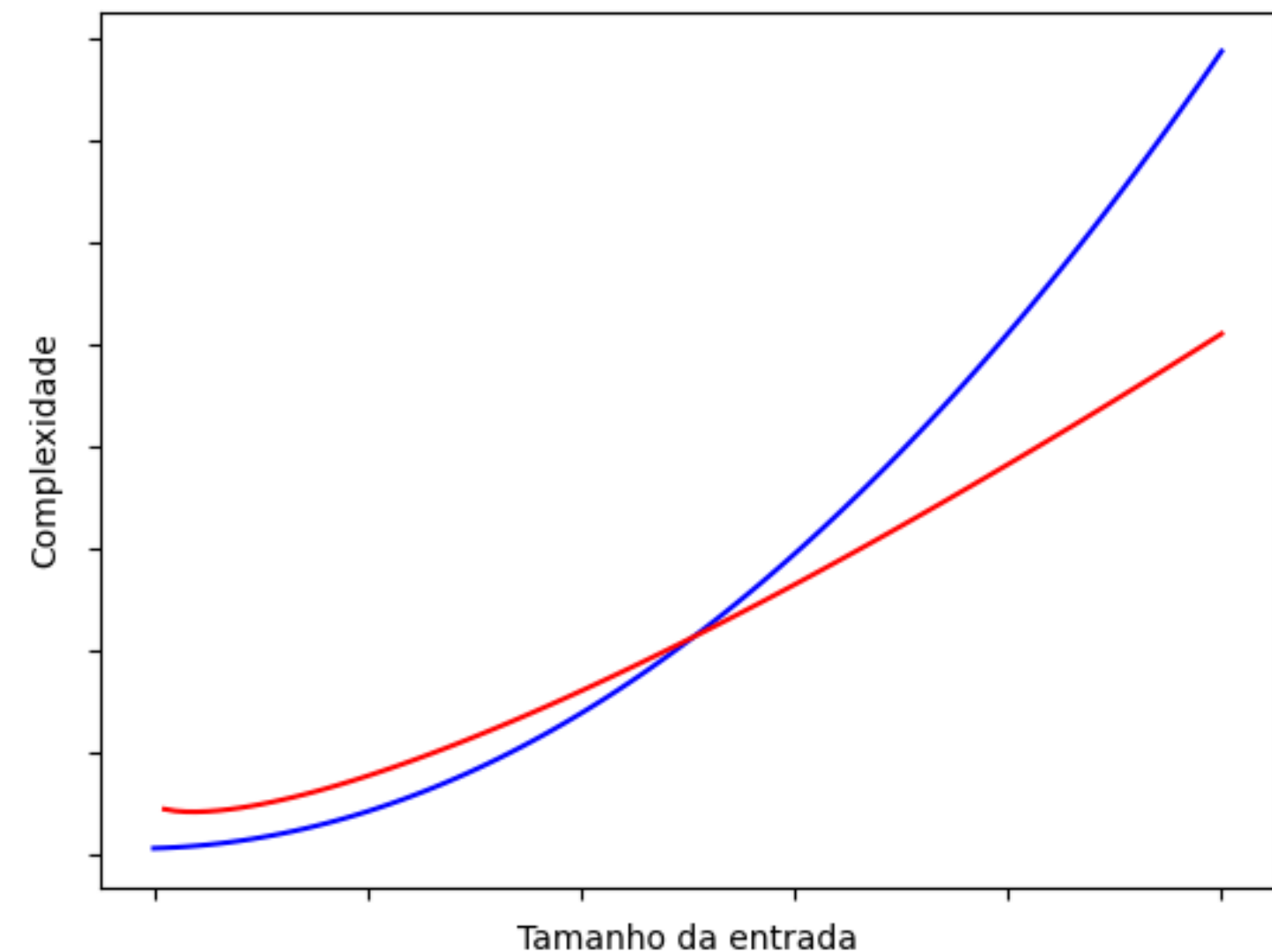
Ordem de crescimento

- A **ordem assintótica de crescimento** de $T(n)$ é definida encontrando as funções que descrevem os limites superior e inferior de $T(n)$.

- **Notação O:**

- **Formal:** $T(n)$ pertence ao conjunto $O(f(n))$ se existirem constantes positivas c e n_0 tais que:

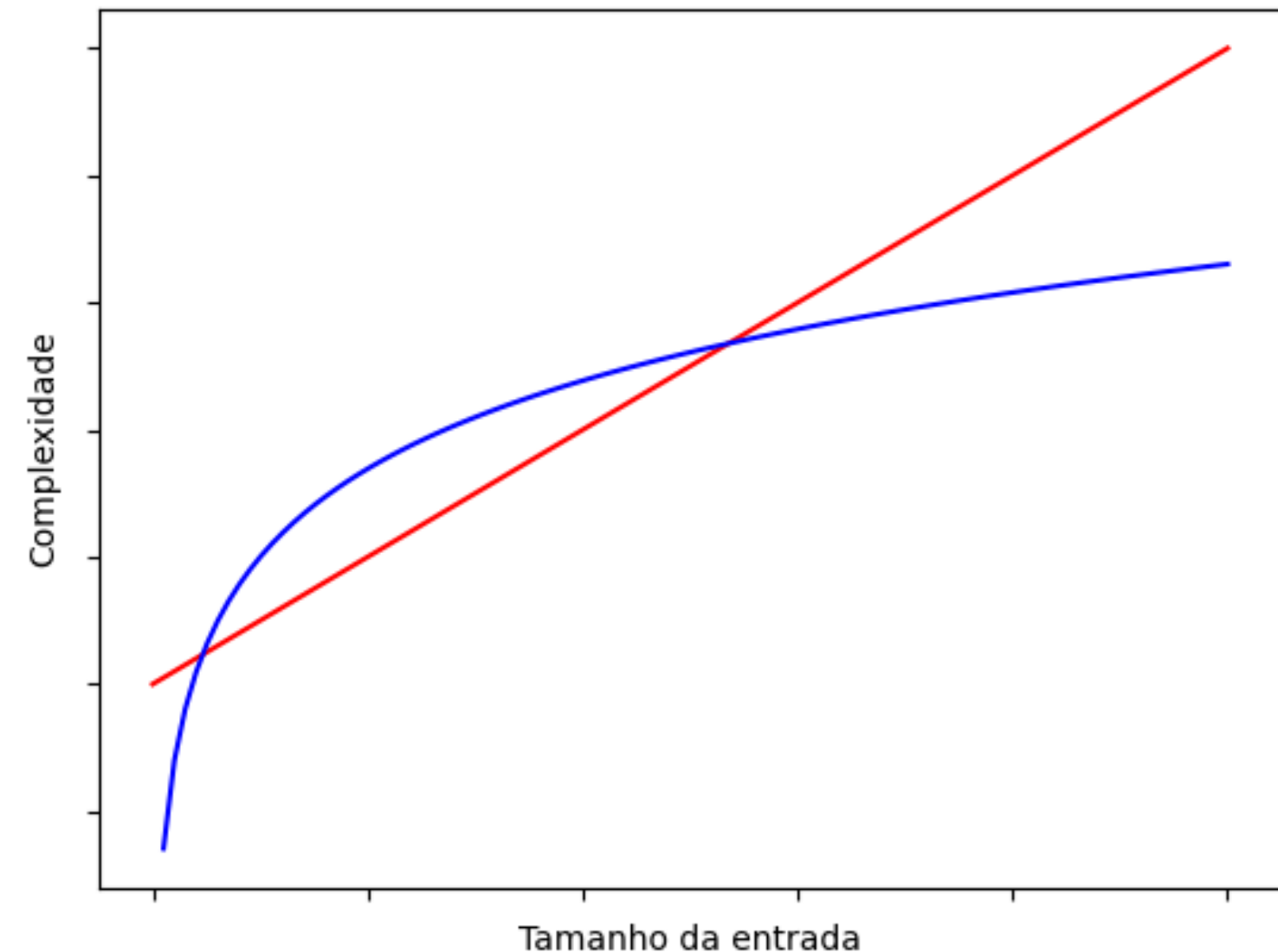
$$T(n) \leq c \cdot f(n) \text{ para todo } n \geq n_0$$



Notação assintótica

Ordem de crescimento

- A **ordem assintótica de crescimento** de $T(n)$ é definida encontrando as funções que descrevem os limites superior e inferior de $T(n)$.
- **Notação Ω :**
 - Informal: $T(n)$ pertence ao conjunto $\Omega(f(n))$ se $T(n)$ crescer no mínimo na mesma ordem de grandeza de $f(n)$.



Notação assintótica

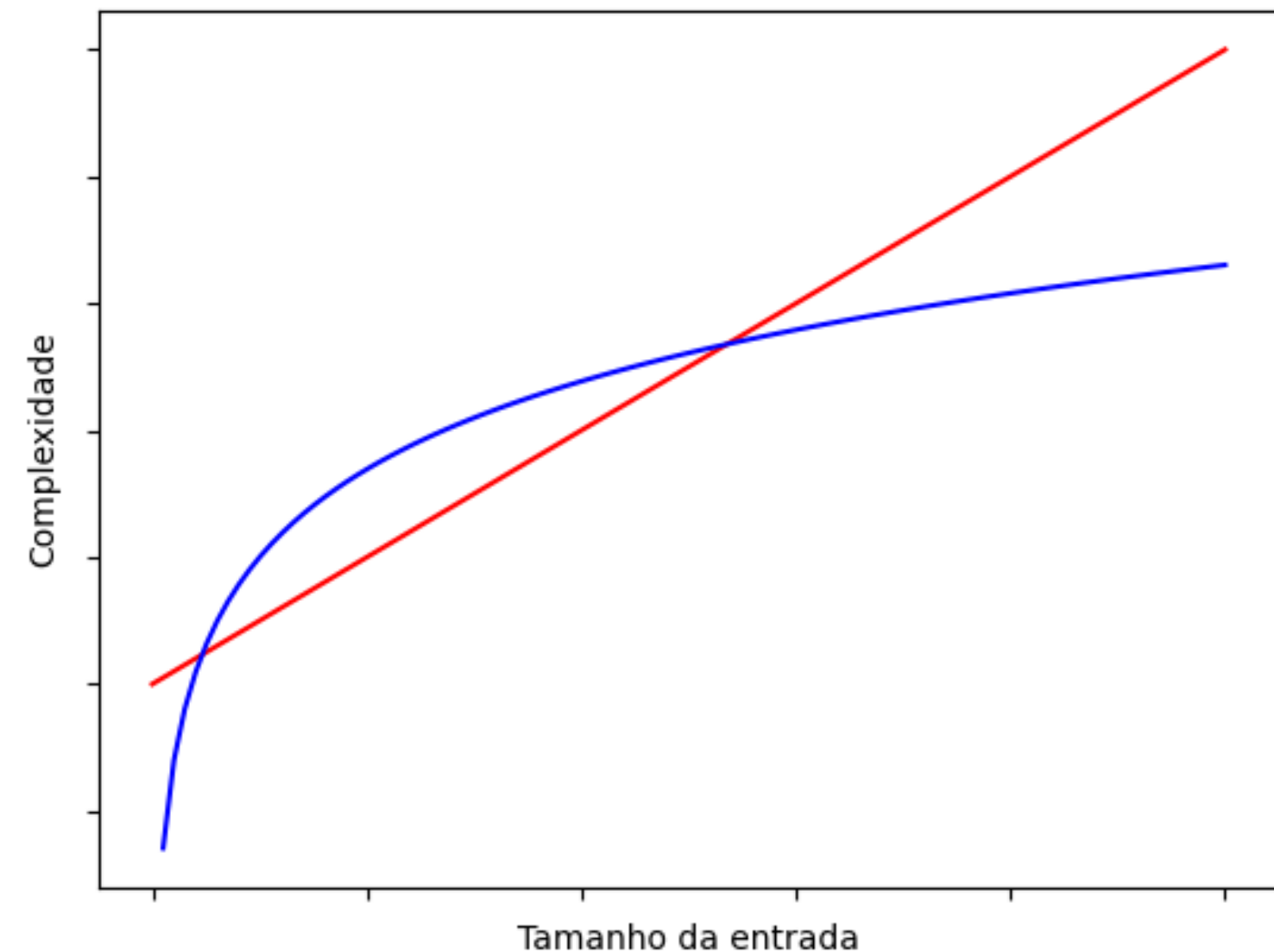
Ordem de crescimento

- A **ordem assintótica de crescimento** de $T(n)$ é definida encontrando as funções que descrevem os limites superior e inferior de $T(n)$.

- **Notação Ω :**

- **Formal:** $T(n)$ pertence ao conjunto $\Omega(f(n))$ se existirem constantes positivas c e n_0 tais que:

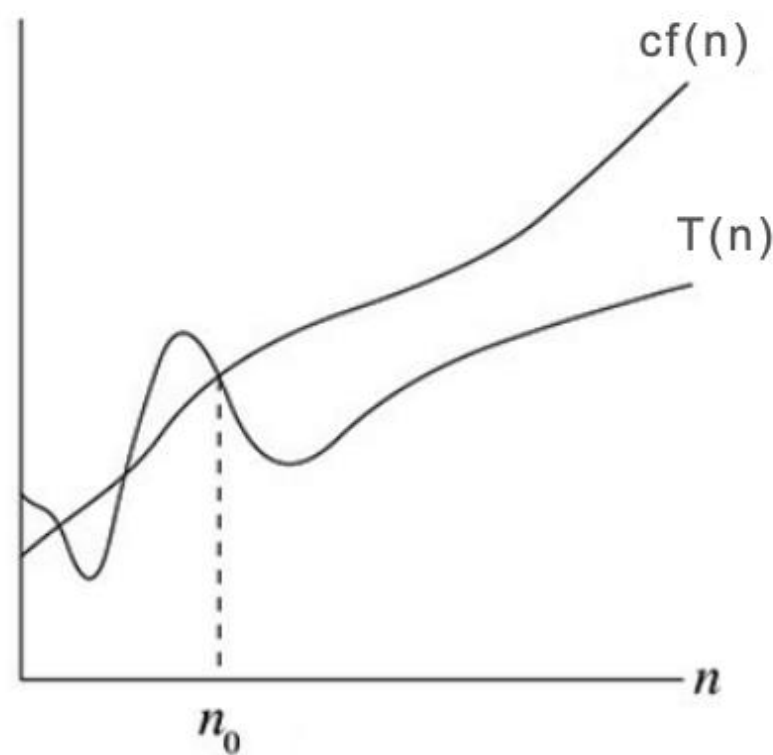
$$c \cdot f(n) \leq T(n) \text{ para todo } n \geq n_0$$



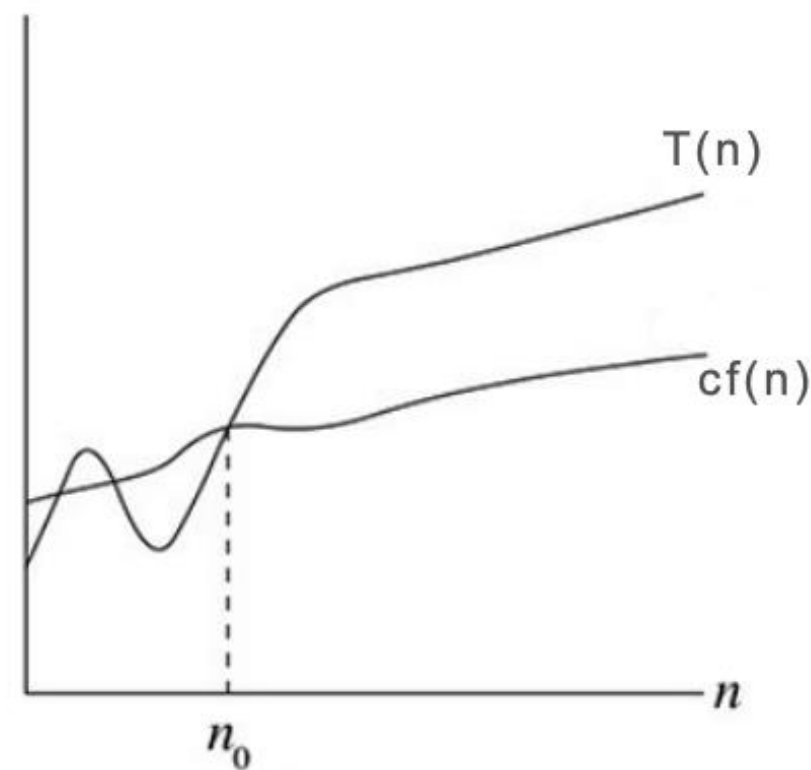
Notação assintótica

Ordem de crescimento

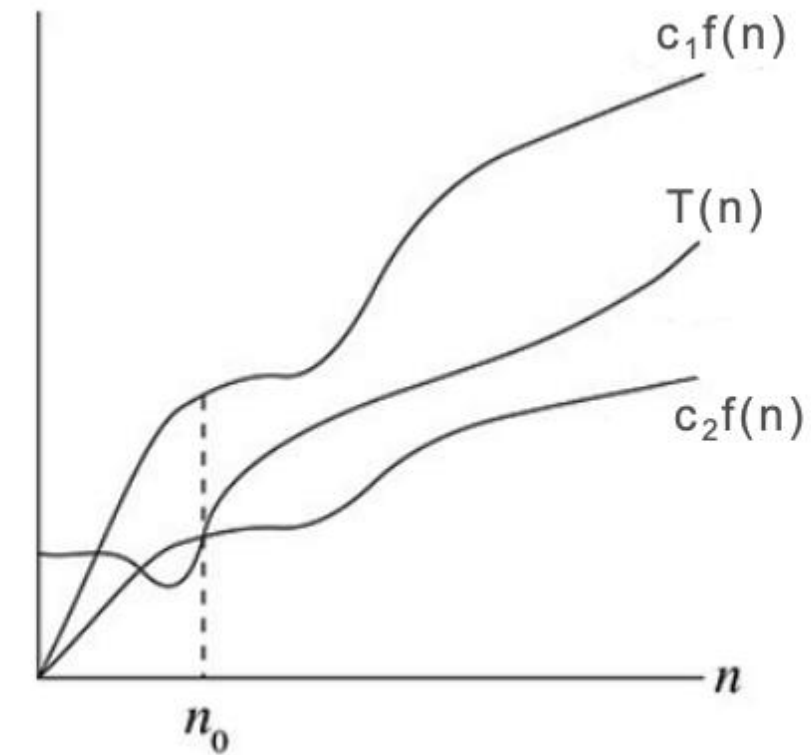
- A **ordem assintótica de crescimento** de $T(n)$ é definida encontrando as funções que descrevem os limites superior e inferior de $T(n)$.
- **Notação Θ :** $T(n)$ pertence ao conjunto $\Theta(f(n))$ se $T(n)$ for $O(f(n))$ e $\Omega(f(n))$.



$$T(n) = O(f(n))$$



$$T(n) = \Omega(f(n))$$



$$T(n) = \Theta(f(n))$$

- **Exercício:** $T(n) = 57n^3 - 20n^2 + 100n + 3072$
 - $O(n^4)$? Sim
 - $\Theta(n^4)$? Não
 - $O(n^3)$? Sim
 - $\Omega(n^3)$? Sim
 - $\Theta(n^3)$? Sim
 - $O(n^2)$? Não
 - $\Omega(n^2)$? Sim
 - $\Theta(n^2)$? Não

- **Lembrando:** múltiplas soluções (algoritmos) podem solucionar o mesmo problema.
 - Essas soluções podem apresentar complexidades diferentes.
- Pergunta: dado um algoritmo qualquer, caso exista uma solução $\Omega(n^2)$ seria possível encontrar uma outra solução para o mesmo problema com complexidade $O(n \log(n))$?
 - Sim! A complexidade é avaliada sobre a solução, não sobre o problema.

Exercícios

- **Exercício 1:** dado $T(n) = 500n^2$, prove que $T(n)$ é $O(n^3)$
- **Exercício 2:** dado $T(n) = n^2 + 423$, prove que $T(n)$ é $O(n^2)$
- **Exercício 3:** dado $T(n) = 10n^3 - 3n^2 + 27$, prove que $T(n)$ é $O(n^3)$
- **Exercício 4:** dado $T(n) = n^2 - 2$, prove que $T(n)$ é $\Omega(n^2)$
- **Exercício 5:** dado $T(n) = 2^n + 1$, avalie se $T(n)$ é $O(2^n)$
- **Exercício 6:** dado $T(n) = 22^n$, avalie se $T(n)$ é $O(2^n)$

- Ao analisar um algoritmo é importante determinar qual o cenário que está sendo considerado:
 - **Pior caso:** é executado o maior número de instruções possível.
 - **Melhor caso:** é executado o menor número de instruções possível.
 - **Caso médio:** média do número de instruções executadas dentre todas as entradas possíveis.
- Quando não for definido explicitamente assuma que estamos utilizando o pior caso.

- **Exemplo:** busca sequencial

- **Pior caso:** o número encontra-se no último elemento da lista ou não está presente na mesma. Complexidade: n .
- **Melhor caso:** o número encontra-se no primeiro elemento da lista. Complexidade: 1.
- **Caso médio:** assumindo que o número encontra-se na lista ($v \in V$) e existe igual probabilidade de estar em qualquer posição ($p_i = 1/n$), temos uma complexidade $(n + 1)/2$.

```
def slow_search(values, value):  
    for i in range(len(values)):  
        if values[i] == value:  
            return i  
    return -1
```

Exemplo de análise

- **Exemplo 1:** qual seria a complexidade assintótica do algoritmo abaixo no pior caso?

```
def select_even(numbers):  
    result = []  
    for number in numbers:  
        if number % 2 == 0:  
  
    result.append(number)  
    return result
```

- Dada uma entrada de tamanho n .
- Consideramos o **pior caso** - todos os números são pares.
- São executadas **no máximo** $3n$ operações $\rightarrow T(n)$ é $O(n)$.
- São executadas **no mínimo** $3n$ operações $\rightarrow T(n)$ é $\Omega(n)$.
- $T(n)$ é $\Theta(n)$

- **Exemplo 2:** qual seria a complexidade assintótica do algoritmo abaixo?

```
def do_something(numbers):  
    intermediate = []  
    for number in numbers:  
        intermediate.append(number * number)  
  
    result = []  
    for number_1 in intermediate:  
  
        if number_1 % 2 == 0:  
            continue  
  
        partial_result = []  
        for number_2 in intermediate:  
            if number_1 % 10 == 0:  
                continue  
            partial_result.append(number_1 * number_2)  
  
        result.append(partial_result)  
  
    return result
```

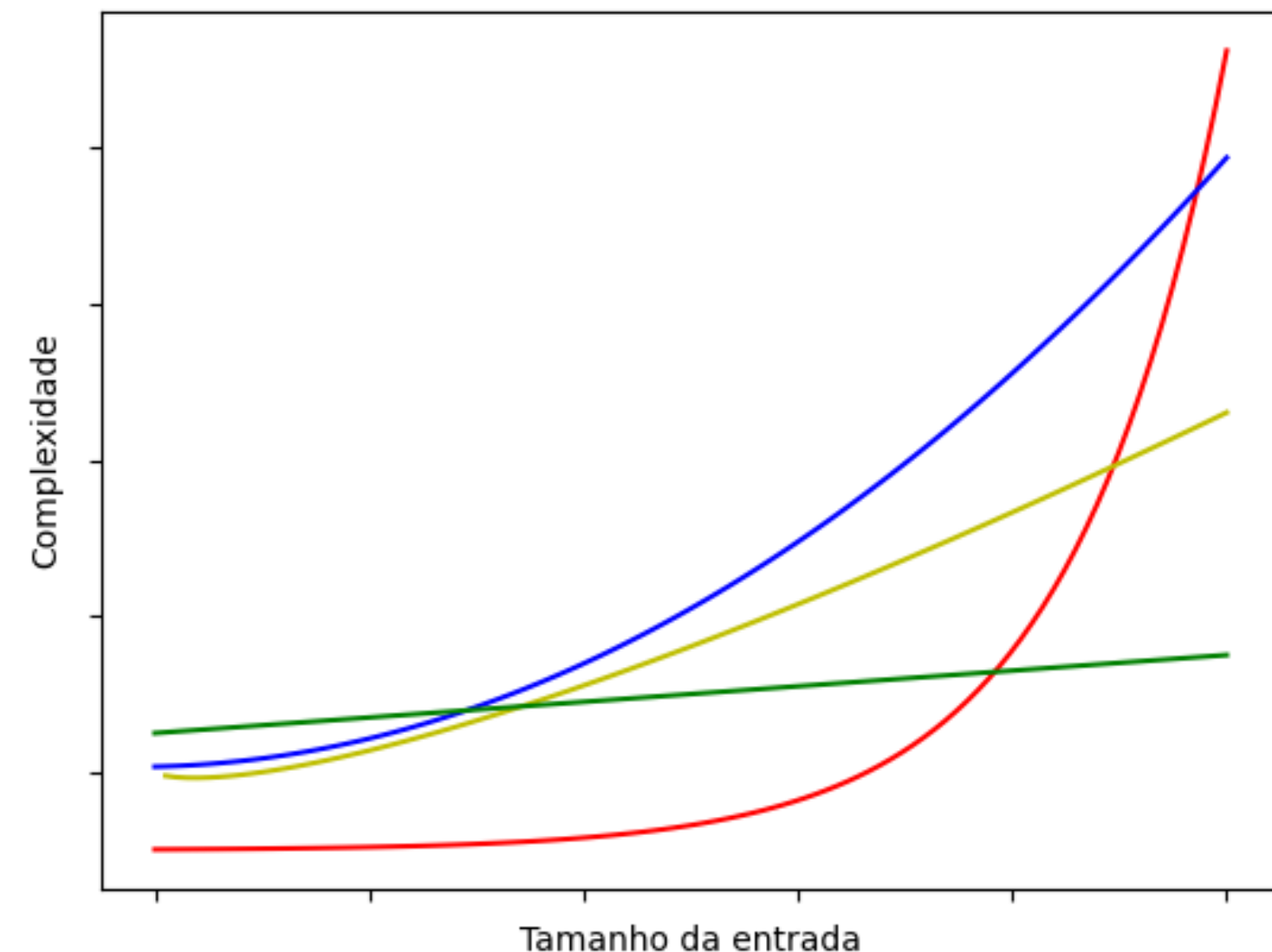
Resposta: $T(n) = \Theta(n^2)$

Notação assintótica

Comparação de algoritmos

- Ao utilizar a notação assintótica **não estamos buscando** determinar com precisão o esforço computacional de um algoritmo.
 - Esperamos entender a taxa de crescimento entre diferentes algoritmos de forma que possamos compará-los.

$$\begin{aligned} T1(n) = 2^n - 100 &\rightarrow \theta(2^n) \\ T2(n) = 10n + 50 &\rightarrow \theta(n) \\ T3(n) = 8n^2 + 3n + 7 &\rightarrow \theta(n^2) \\ T4(n) = 20n \log(n) &\rightarrow \theta(n \log(n)) \end{aligned}$$



Comparação de algoritmos

- Suponha que os algoritmos a seguir solucionam o mesmo problema. Qual seria mais eficiente para uma entrada muito grande?

```
def do_something_1(numbers):  
    result = []  
    for number_1 in numbers:  
        for number_2 in numbers.reverse():  
            size = len(numbers) // 2  
            for number_3 in numbers[:size]:  
                value = execute_operation(  
                    number_1, number_2, number_3  
                ) # Constant  
                result.append(value)  
  
    return result
```

$$T_1(n) = \Theta(n^3)$$

```
def do_something_2(numbers):  
    result = []  
    for number_1 in numbers:  
        value = execute_pre_operation(number)  
        result.append(value)  
  
    for number_1 in result:  
        for number_2 in numbers.reverse():  
            value = execute_operation(  
                number_1, number_2  
            ) # Constant  
            result.append(value)  
  
    return result
```

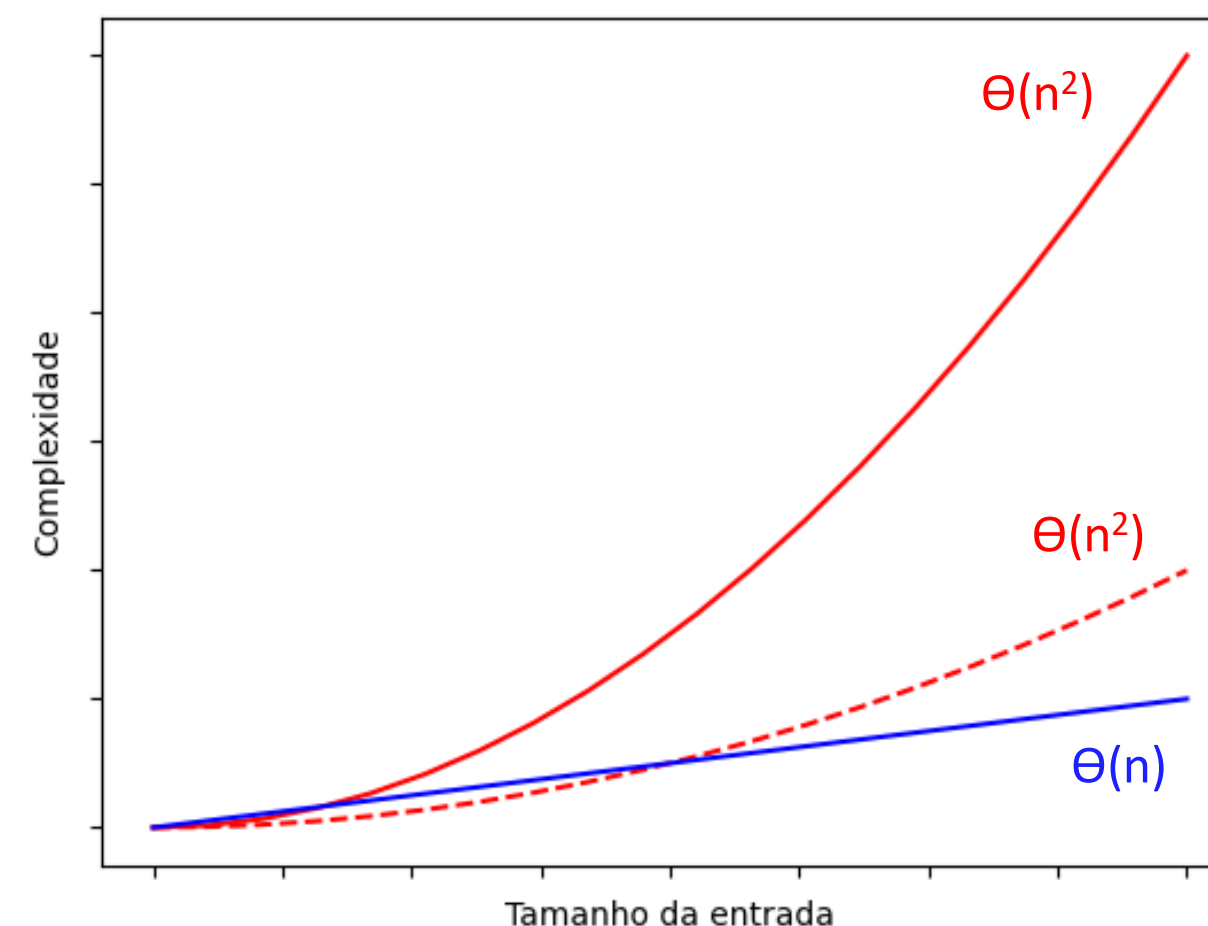
$$T_2(n) = \Theta(n^2)$$

- Resposta: $T_2(n) = \Theta(n^2)$

Notação assintótica

Comparação de algoritmos

- Poderíamos apenas otimizar a implementação, sem alterar o algoritmo?
 - Ex: “*otimizamos um algoritmo $\Theta(n^2)$ substituindo algumas instruções por alternativas mais eficientes. A execução tornou-se 10x mais rápida. Seria suficiente para evitar a implementação de um algoritmo $\Theta(n)$? Considere que iremos utilizar o algoritmo com volumes massivos de dados.*”
- Não, pois a taxa de crescimento do algoritmo $\Theta(n^2)$ cuja implementação foi otimizada continua sendo maior que a do algoritmo $\Theta(n)$.



Notação assintótica

Comparação de algoritmos

- $\theta(1)$ Constante
- $\theta(\log n)$ Logarítmica
- $\theta(n)$ Linear
- $\theta(n^2)$ Quadrática
- $\theta(n^3)$ Cúbica
- $\theta(nk)$ com $k \geq 1$ Polinomial
- $\theta(2^n)$ Exponencial
- $\theta(an)$ com $a > 1$ Exponencial
- *Outras ...*

Notação assintótica

Comparação de algoritmos

- Considere os tempos (hipotéticos) de algoritmos com diferentes complexidades:

Tamanho da entrada	n	$n \log n$	n^2	n^3	1.1^n	2^n	$n!$
100.000	< 1s	< 1s	< 1s	< 1s	< 1s	5s	10min
1.000.000	< 1s	< 1s	< 1s	< 1s	5min	22min	meses
100.000.000	< 1s	< 1s	11s	55s	22d	meses	anos
1.000.000.000	2s	8s	16min	3h	meses	anos	anos
100.000.000.000	3min	5min	5h	2d	anos	anos	anos

- Buscamos encontrar soluções com o tempo polinomial!

Perguntas?

`thiago.p.araujo@fgv.br`